

Lesson 12: Hearing It Happen

Godot Engine

VIDEO GAME CREATION

This week: your game finally sounds like a game. Four moments - firing, hitting, clearing a wave, and losing - each get their own distinct cue.

Open your own [YourName]Game project from Lesson 11 - don't start a new one.

If you already built an optional sound idea in an earlier lesson (Lessons 2, 3, 4, or 6), keep it - today reuses it directly instead of replacing it. If you didn't, today's steps are written as if starting fresh.

Setup

1. Open your [YourName]Game project from Lesson 11.
2. Add a new AudioStreamPlayer2D node to your main scene, named RoundSound - one shared player for the round-ending sounds (win, wave-clear, lose), since only one of them ever plays at a time.
3. Import sfx_wave.wav and sfx_lose.wav (Resources > Import > Sound) alongside your existing sfx_shoot.wav, sfx_hit.wav, and (if you built it) sfx_win.wav.

Level 1 - Minimal (MVP)

Four sounds fire at the correct moment, at least once each: a shot, a hit/explosion, a wave-clear (on any non-final wave), and a game over. No silence, no obviously wrong triggers.

4. On the Player, confirm $AudioStreamPlayer2D.play() already fires in your fire function next to bullet instantiation (from Lesson 3) - add it now if it's missing.
5. On the Enemy scene's script, confirm its own AudioStreamPlayer2D plays sfx_hit.wav when destroyed (from Lesson 4) - add it now if it's missing.
6. At the top of your main script, preload SND_WAVE and SND_LOSE constants (and SND_WIN, if you have it).
7. In _on_enemy_destroyed()'s nested if (from Lesson 11), assign RoundSound's stream and call .play() in each branch - SND_WIN for the true-win case (skip if you didn't build it), SND_WAVE for the wave-advance case.
8. In _on_enemy_escaped()'s lose branch, assign RoundSound's stream to SND_LOSE and call .play().
9. Run the scene and play through a full round - shoot, hit, an early wave clear, and a loss should each sound distinct.

```
# Top of main script:
const SND_WAVE = preload("res://sfx_wave.wav")
const SND_LOSE = preload("res://sfx_lose.wav")
const SND_WIN = preload("res://sfx_win.wav")  # only if you built Lesson
6's optional idea

func _on_enemy_destroyed():
    enemies_remaining -= 1
    score += 10
    update_hud()
    if enemies_remaining <= 0:
        if wave >= total_waves:
            game_active = false
            $WinText.show()
            $RoundSound.stream = SND_WIN
            $RoundSound.play()
        else:
            wave += 1
            enemies_remaining = 5
            update_hud()
            $RoundSound.stream = SND_WAVE
            $RoundSound.play()
            spawn_wave()

func _on_enemy_escaped():
    lives -= 1
    update_hud()
    if lives <= 0:
        game_active = false
        $LoseText.show()
        $RoundSound.stream = SND_LOSE
        $RoundSound.play()
```

Level 2 - Complete (Alpha/Beta)

All four sounds fire reliably every time, with no missing or duplicate playback, and each is clearly distinguishable from the others by ear.

- Play a full round: shoot, hit, an early wave clear, and a loss each produce a distinct sound.
- If you built the optional win sound, confirm it still plays on the true win, and SND_WAVE doesn't also fire on that same frame.
- Confirm you're setting .stream immediately before every .play() call on RoundSound, not relying on whatever it was set to last.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Audio: if you haven't already, add the optional win sound now - it slots into the exact same $RoundSound.play() pattern as today's wave/lose sounds, just a third constant.
- Audio: add slight pitch variation to hits by setting the Enemy's AudioStreamPlayer2D pitch_scale to a random value (e.g. randf_range(0.9, 1.1)) right before each .play() call.
- Structure: notice RoundSound is shared by three different outcomes (win, wave-clear, lose) - one node, three interchangeable clips, instead of three separate always-idle nodes.

If Something's Wrong

- Wrong sound plays: confirm .stream is being reassigned immediately before each .play() call, not left over from a previous event.
- No sound on wave-clear: confirm the SND_WAVE play() call is actually inside the else branch, not after the if/else.
- Lose sound plays repeatedly: confirm the lose branch is still guarded by "if lives <= 0" as a one-time check (from Lesson 8), not re-triggering every frame lives stays at 0.

Want More?

Student Resource List > Godot section, or docs.godotengine.org